Programowanie współbieżne w C++

Lista zadań nr 2

Na pracownię 12 marca 2026

Zadanie 1 (3pkt). Wykonaj refaktoryzację Zadania 1 z Listy 1 w taki sposób, że: a) ograniczony bufor wraz z funkcją `produce` i `consume` opakowany będzie w klasę, b) klasa ta jest sparametryzowana dowolnym typem T, będącym typem wartości produkowanych i konsumowanych, oraz c) obiekty tej klasy nie są kopiowalne ani przenaszalne[1]. Funkcje `produce` i `consume` powinny obsługiwać wkładanie/wyciąganie elementów z bufora i **usypianie** wątku wywołującego w razie potrzeby. Upewnij się, że Twoja symulacja wątków producentów i konsumentów nadal działa poprawnie.

Zadanie 2 (5pkt). Zaimplementuj własną wersję semafora zliczającego (na wzór std::counting_semaphore). Semafor powinien dostarczać metody:

```
void release( std::ptrdiff_t update = 1 );
void acquire();
bool try_acquire() noexcept;
```

Jako mechanizmy synchronizacji użyj `std::mutex` oraz `std::condition_variable`. Przetestuj swoje rozwiązanie.

Zadanie 3 (5pkt). Przebuduj semafor z poprzedniego zadania tak, by implementował również metodę:

```
acquire_interruptible(std::stop_token stoken)
```

umożliwiającą przerwanie oczekiwania po wysłaniu sygnału. Przetestuj swoje rozwiązanie, np. w połączeniu z `std::jthread`.

Zadanie 4 (5pkt). Zaimplementuj wielowątkowy silnik dla "Gry w Życie" Conwaya. Plansza to dwuwymiarowa tablica komórek (żywa/martwa). Gra ma ustaloną liczbę iteracji. W każdej iteracji stan planszy zależy wyłącznie od stanu planszy z poprzedniej iteracji, według reguł gry. Początkowy stan

[1] Zatem, jeśli potrzebujesz przekazywać obiekty tej klasy do funkcji, przekazuj je przez referencje.

planszy jest losowy. Z racji tego, że przejścia stanów można liczyć niezależnie dla każdej komórki, problem idealnie nadaje się do zrównoleglenia poprzez podział planszy, np. na sektory poziome. Za obliczenie nowego stanu każdego sektora tablicy będzie odpowiadać osobny wątek. Oto szkielet rozwiązania:

```
class GameOfLife {
public:
    // Struktura definiująca krok zakańczający barierę.
    struct CompletionStep {
        GameOfLife* game_instance;
        void operator()() noexcept {
            game_instance->end_of_phase_action();
        }
    };

private:
    int width, height;
    int num_threads;
    int num_iterations;

    // Płaskie wektory symulujące planszę 2D (0 - martwa, 1 - żywa)
    std::vector<uint8_t> grid_a;
    std::vector<uint8_t> grid_b;

    // Wskaźniki do aktualnej planszy odczytu i zapisu
    std::vector<uint8_t>* read_grid;
    std::vector<uint8_t>* write_grid;

    // Grupa wątków roboczych
    std::vector<std::jthread> workers;

    // MIEJSCE NA TWOJĄ BARIERĘ:
    // std::barrier<CompletionStep> ...

    // --- METODA POMOCNICZA ---
    // Przelicza współrzędne (x, y) na płaski indeks 1D z uwzględnieniem
zawijania krawędzi (torus)
    inline int get_index(int x, int y) const {
        int wrapped_x = (x + width) % width;
        int wrapped_y = (y + height) % height;
        return wrapped_y * width + wrapped_x;
    }

    // Metoda wywoływana dokładnie raz na koniec każdej iteracji
    void end_of_phase_action() noexcept {
        std::swap(read_grid, write_grid);
        // Tutaj można też bezpiecznie zaktualizować globalne zmienne bez
użycia mutexów
    }

    // Funkcja wykonywana przez każdy z wątków (oblicza komórki od start_row
do end_row-1)
    void worker_thread(int thread_id, int start_row, int end_row);

    // Funkcja pomocnicza: zliczanie żywych sąsiadów dla komórki (x, y)
    int count_alive_neighbors(int x, int y) const;

public:
```

```
    GameOfLife(int w, int h, int threads, int iterations);

    // Inicjuje planszę początkowym stanem i uruchamia wątki
    void run();

    // Rysuje planszę w konsoli
    void print_grid() const;
};
```

Szkielet można modyfikować według potrzeb. Wymiary planszy należy podać w konstruktorze obiektu. Symulacja gry rozpoczyna się po uruchomieniu metody `run()`. Tam też należy wylosować stan początkowy. W każdej iteracji czytamy z planszy wskazywanej przez `read_grid` i zapisujemy do planszy wskazywanej przez `write_grid`. Dostęp do tych plansz jest wątkowo bezpieczny i nie wymaga dodatkowej synchronizacji (z jednej tylko czytamy, do drugiej piszemy w rozłącznych komórkach)[2] Po każdej iteracji planszę należy wypisać na ekranie i zamienić `read_grid` z `write_grid`. Należy to zrobić wewnątrz funkcji `end_of_phase_action()` wcześniej zapewniając przy pomocy bariery, że ta funkcja wykona się na końcu każdej iteracji.

[2] Użycie `bool` jako typu komórek planszy uczyniłoby to zdanie nieprawdziwym. Specjalizacja szablonu `std::vector<bool>` może umieszczać różne elementy wektora w jednym bajcie pamięci, tym samym powodując wyścig przy modyfikacjach w różnych wątkach. Ponadto, nawet dla zastosowanego typu `std::vector<uint8_t>` jakakolwiek operacja modyfikująca rozmiar tablicy (np `push_back()`) jest wątkowo niebezpieczna.